Building a Deephaven Application: A Programmer's Guide

Oct 21, 2026 · @Taylor

Read this first

Deephaven is a live spreadsheet engine with a server attached, not an event-loop framework. In QuantConnect LEAN you write callbacks (`Initialize` , `OnData`) that the engine calls once per time slice. In Deephaven, your script runs **once**. It declares tables and how each is computed from the others. From then on, the engine recomputes everything that changed, about once a second by default, and pushes the changes to every UI and client.

You write four kinds of thing. Everything else belongs to the engine:

1. **Sources:** where rows come from (a clock, Kafka, a Python feed you push into, a file, a table another worker shares, a form a user fills in).
2. **The graph:** table operations that derive tables from tables (filters, formulas, joins, aggregations, rolling windows). This is most of the program, and it is declarative.
3. **Edges:** the few places where Python code runs on changes (listeners), plus sinks that write out.
4. **Surfaces:** what other people see and touch (named tables, deephaven.ui dashboards, input tables, client APIs).

The rule of thumb that makes it click: if you would write it as a spreadsheet formula, it belongs in the graph; if you would write it as a macro, it belongs in a listener, and you should have very few. A trading app built this way has no `OnData` . Signals, positions and risk are tables that are always current, and orders are rows that a listener sends out.

How to read this guide:

- **Anatomy** shows what actually runs, on which thread, in what order.
- **Programming model** is the construct catalog with exact syntax.
- **Extending** and **Backtesting** answer what you can add and how history replays.
- **Worked engine** shows one program end to end.
- **Deephaven vs. LEAN** maps the concepts and states the limits.

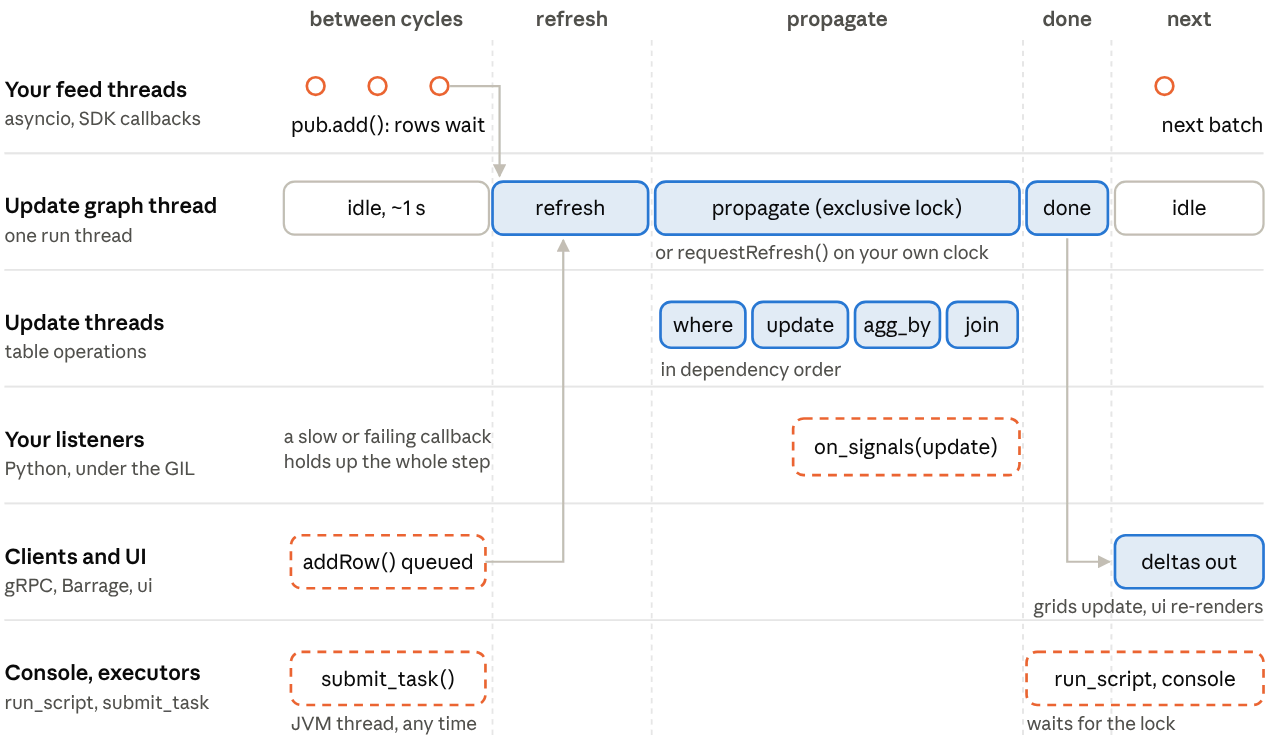
Claims marked **Tested** ran on Community 42.6 in this repo. **Docs** means read in Deephaven's documentation, and **Source** means read in the installed 42.6 code.

Anatomy of a running app

A running app is one engine thread stepping the graph about once a second, while your threads and the clients only queue work for the next step. Nothing you push is visible until a refresh picks it up, and nothing a client sees is ever half a step.

One cycle across threads: your rows wait, one step runs, then clients see it

Left to right in time, default update graph. Filled boxes are the engine; dashed boxes are your code.

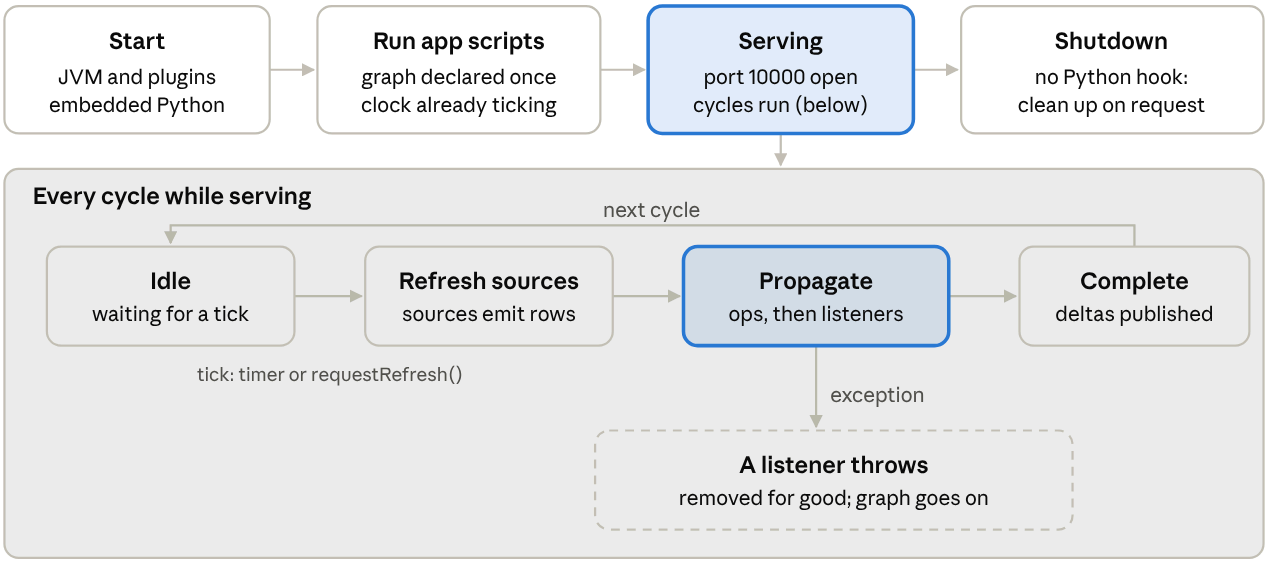


Rows pushed mid-cycle wait for the next refresh; clients always see a whole step, never half a cycle.

thread swimlanes · one cycle of the default update graph

Each lane is a kind of thread in one worker. The feed thread's rows and the client's `addRow()` wait in buffers until the refresh. The table operations, and then your listeners, run inside the step while the lock is held. Only after the step completes do deltas go out to grids and deephaven.ui. Console commands wait for the lock; `submit_task` work runs on its own JVM threads at any time.

The engine state machine: built once, then one cycle after another



engine state machine · lifecycle and the per-cycle loop

The top row happens once per worker. The loop below it repeats for the life of the worker. A listener that throws drops out of the loop permanently, while the rest of the graph carries on, so wrap every callback.-$

Thread	Who owns it	What runs there	Has an execution context
Thread that starts the server	engine	startup, application-mode scripts	yes
Update graph run thread and update threads	engine	refresh, table operations, your listeners	yes
Script session	engine	console and `run_script` commands	yes
Server executors	engine	`submit_task("serial" | "concurrent", fn)`	yes
gRPC and Barrage threads	engine	client requests, subscriptions, deltas	n/a

Thread	Who owns it	What runs there	Has an execution context
Your threads (feeds, asyncio)	you	anything; `pub.add(...)` is safe from here	only if you pass one in (`with ctx:`)

The programming model: every construct and its syntax

There is no special DSL file. A Deephaven program is an ordinary Python script calling the `deephaven` package. A small Java-like expression language appears inside quoted strings (formulas and filters), and the engine compiles each string to Java. Signatures below are from the installed 42.6 package.

1. Sources: where rows come from

Construct	Syntax	Use it for	Evidence
Clock	`time_table("PT0.25s")`	timers, heartbeats, scheduled work	Tested
Push feed	`t, pub = table_publisher("quotes", {"Sym": dht.string, "Px": dht.double})` then `pub.add(new_table(...))` from any thread	**the generic adapter**: wrap any client (websocket, FIX, Solace SDK, asyncio)	Tested
Polled function	`function_generated_table(fn, refresh_interval_ms=1000)` or `source_tables=[t]`	re-run your Python function that returns a whole table (REST snapshots, small state)	Tested
User / client input	`input_table({"Desk": dht.string, "Limit": dht.double}, key_cols="Desk")`	limits, overrides, commands; keyed = upsert, `delete` by key	Tested
Kafka	`kafka_consumer.consume(cfg, "trades", key_spec=..., value_spec=..., table_type=TableType.blink())`	broker feeds	Docs
Files and lakes	`read_csv` , `parquet.read` , `deephaven.experimental.iceberg`	history, reference data	Docs

Construct	Syntax	Use it for	Evidence
Another worker	`deephaven.uri.resolve("dh+plain://host:10000/scope/positions")`; Enterprise `pq://RiskEngine/scope/positions`, `remote_table`	sharing between apps	Tested (dh+plain), Docs (pq://)
Enterprise tables	`db.live_table("MarketUs", "Quotes")`, `db.historical_table(...)`	the plant's intraday and historical store	Docs
Replay	`TableReplayer(start, end).add_table(hist, "Timestamp")`	history as if live (see Backtesting)	Tested
Windows over a feed	`ring_table(blink, capacity=10_000)`, `blink_to_append_only(blink)`	keep the last N rows, or everything	Tested

A push feed produces a **blink** table: each cycle it holds only the rows added in that cycle. Aggregations over a blink table therefore accumulate correctly without holding history in memory.

2. The graph: tables derived from tables

Every call returns a new live table; nothing runs again until an input changes.

```
mid    = quotes.update(["Mid = (Bid + Ask) / 2", "Spread = Ask - Bid"])
# formulas (compiled to Java)
liquid = mid.where(["Spread < 0.05", "Sym in `AAPL`, `MSFT`"])
# filters
bars   = trades.update("Minute = lowerBin(Timestamp, MINUTE)") \
               .agg_by([agg.first("Open = Px"), agg.max_("High = Px"),
agg.min_("Low = Px"),
                        agg.last("Close = Px"), agg.sum_("Volume = Qty")],
by=["Sym", "Minute"])
ema    = mid.update_by(ema_time("Timestamp", "PT5m", "Ema = Mid"), by="Sym")
# rolling / cumulative ops
filled = trades.aj(quotes, on=["Sym", "Timestamp"], joins=["Bid", "Ask"])
# as-of join: quote at trade time
book   = positions.natural_join(mid, on="Sym", joins="Mid") \
                  .update("Mtm = Qty * Mid")
# lookup join
risk   = book.agg_by([agg.sum_("Mtm"), agg.sum_("Dv01")], by="Desk")
# roll-ups
latest = quotes.last_by("Sym")
# current state per key
per_sym = trades.partition_by("Sym")
# one sub-table per key
snap   = book.snapshot_when(time_table("PT1m"))
# sample on a schedule
```

`update_by` offers about 40 operations: `ema_*`, `ems_*`, `cum_*`, `delta`, `forward_fill`, and `rolling_{sum,avg,min,max,std,wavg,count,prod,group,formula}_{tick,time}`. `rolling_formula_*` takes your own expression over each window.

3. Edges: where your Python runs

```
def on_signals(update, is_replay):              # runs on an engine update
thread, inside the cycle
    for row in update.added().get("Sym", []):
        send_order(row)                          # keep it short; failures
stop the listener for good
h = listen(signals, on_signals, on_error=log_failure)
m = merged_listen([quotes, orders, signals], on_checkpoint)   # one call per
cycle, all tables at one step
h.stop()                                         # always keep the handle and
stop it on reload

submit_task("concurrent", heavy_job)            # deephaven.server.executors:
run Python on a server JVM thread
kafka_producer.produce(orders_tbl, cfg, "orders", key_spec, value_spec)    #
sink (Docs)
parquet.write(snapshot_tbl, "/data/eod.parquet")                            #
sink
```

To keep state across cycles, a listener writes rows back into a `table_publisher`. That loop is how custom stateful logic re-enters the graph.

4. Clocks, threads and contexts

Construct	Syntax	Evidence
Default clock	about 1 s cycles; set `-DPeriodicUpdateGraph.targetCycleDurationMillis`	Tested
Your own clock	`EventDrivenUpdateGraph.newBuilder("algo").build()`; build tables under `ExecutionContext(...withUpdateGraph(ug))`; step with `ug.requestRefresh()`	Tested
Hand the context to your thread	`ctx = get_exec_ctx()` on a thread that has one; then `with ctx:`	Tested
Read consistently from outside the cycle	`with update_graph.shared_lock(t):` or `exclusive_lock`	Source

5. Surfaces: what people and other programs see

```
quotes_view = mid            # any global name holding a table is a field
clients can open

get_app_state()["risk"] = risk   # in an app.d/*.app script: a field under
that app's id
                                 # (from deephaven.appmode import
get_app_state)

@ui.component                # deephaven.ui: React-style components rendered
from Python
def desk_view():
    desk, set_desk = ui.use_state("Rates")
    summary = ui.use_row_data(risk.where(f"Desk == `{desk}`"))
    return ui.flex(ui.picker("Rates", "FX", selected_key=desk,
on_change=set_desk),
                   ui.table(book), dx.line(ema, x="Timestamp", y="Ema",
by="Sym"))
dashboard = ui.dashboard(desk_view())
```

6. Clients: how other programs talk to it

All clients use one port (10000) and the same three moves: read a table by name, run code, write rows.

Client	Read live data	Write	Run code	Evidence
Python `pydeephaven`	`s.open_table("risk").to_arrow()` (snapshot)	`s.input_table(...).add(t)`, `s.bind_table(name, t)`	`s.run_script(code)`	Tested
JavaScript `@deephaven/jsapi` (React)	`table.setViewport(0, 49)` + `EVENT_UPDATED` (ticking)	`table.inputTable().addRow({...})`	`ide.runCode(code)`	Tested
Another worker	`uri.resolve("dh+plain://...")`, `pq://` (ticking)	through its input tables	none	Tested / Docs
Java, C++, R, Go, Flight SQL	Barrage subscriptions	input tables	console sessions	Docs

A client never sees Python objects, only exported tables and widgets, through tickets. Ticking reads are Barrage subscriptions: a snapshot, then per-cycle deltas.

Extending the framework

You can add your own functions at both the Python and the Java level, and your own data sources. You cannot plug in a new incremental table operation through a public API. New operations are built by composing the built-in ones, or from a listener that feeds a publisher.

Extension	How	Cost and limits	Evidence
Python function in a formula	`def pnl(qty: float, px: float, mark: float) -> float: ...` then `update("PnL = pnl((double) Qty, Px, Mark)")`	Runs per row, under the GIL. Type hints give a typed column; without hints the column is a generic `PyObject`	Tested
Java function in a formula (the optimized path)	Compile a class, add the jar with `Server(extra_classpath=[...])` (or the worker classpath on Enterprise), then call `com.acme.Risk.dv01(Qty, Px, Dur)` in a formula	Runs as compiled Java per row, with no Python crossing	Tested
Your own operation library	Python functions that take tables and return tables, e.g. `def vwap(t, by): return t.agg_by(...)`	Each built-in inside stays incremental	Tested (the same `build_graph` ran live, under replay and in a backtest)
Custom rolling window	`rolling_formula_time("Timestamp", formula="...", rev_time="PT5m")`	An expression over each window, incremental	Source
Stateful custom logic	A listener computes and writes into a `table_publisher`, which feeds the graph	Python cost per cycle; you own the state	Tested
Custom upstream source, Python	`table_publisher`: any thread calls `pub.add(table)`; rows land at the next cycle	The generic adapter for any client SDK	Tested
Custom upstream source, Java	Implement `io.deephaven.stream.StreamPublisher` and feed it through `StreamToBlinkTableAdapter` (the path Kafka uses)	Chunk-level and the fastest, but you write Java against engine interfaces	Docs (Javadoc)

Extension	How	Cost and limits	Evidence
Custom stored-data backend	`deephaven.experimental.table_data_service`	Serves lazily loaded, partitioned history from your own store	Source (experimental)
New object types and widgets	`ObjectType` / `BidirectionalObjectType` plugins (serialize your objects to clients, two-way message streams) plus JS plugins for the browser	deephaven.ui and plotly-express are built this way	Source
A genuinely new table operation (e.g. a custom join)	No public SPI. Possible only in Java against engine internals: `QueryTable`, `TableUpdateListener`, `RowSet`, `ModifiedColumnSet`	Unsupported, tied to the engine version; ask the vendor first	Source

Three gotchas we hit on native Windows:

1. **Integer type hints fail.** `def f(qty: int)` and `np.int64` are rejected with "Unsupported type hint". Deephaven expects numpy code `l` for 64-bit integers, but on Windows numpy reports int64 as `q`. Use float hints with a `(double)` cast, or run on Linux.
2. **`query_library.import_static("com.acme.Risk")` fails** with ClassNotFound for a jar added through `extra_classpath`, although the JVM loads the class. Call by fully qualified name, or import the Class object: `get_exec_ctx().j_exec_ctx.getQueryLibrary().importStatic(jpy.get_type("com.acme.Risk").jclass)`.
3. **An unhinted Python function** returns a `PyObject` column, which formulas downstream cannot do arithmetic on without a cast.

Where each kind of logic belongs, in order of preference:

1. a built-in table operation
2. a Java function in a formula
3. a Python function in a formula
4. a listener and publisher loop

Each step down adds per-row or per-cycle cost and moves work onto the GIL.

Backtesting and replay

Yes, with one condition: write the graph as a function of its source tables. Then the same graph code runs live, under replay, or in a fast backtest, and only the sources you pass in change. Deephaven offers three ways to feed history in:

	`TableReplayer` (Community)	Replay query (Enterprise)	Event-driven backtest (Community)
What it does	Releases rows of a historical table by their timestamp	Runs a whole PQ against a past date with a simulated clock	Your loop pushes each historical slice and steps the graph
Speed	Wall-clock time only (no speed setting in the Python API)	Configurable (0.5x, 2x, ...)	As fast as the engine computes
Script changes	Swap the source tables	**None**, if the script uses `db.live_table` and built-in date functions; hard-coded dates must change	Swap the sources; bind the graph to your clock
`now()`, `today()`, `time_table`	Stay on wall-clock time	Follow the simulated clock	Stay on wall-clock time
Measured	20 rows over 4 s released at about 5 per second; graph result identical; `now()` was wall time	Not testable here	200 slices in 0.137 s, one consistent step and one listener call each; `requestRefresh()` returned only after the step finished (200 of 200)
Evidence	Tested	Docs	Tested

```
def build_graph(trades, quotes):                # the strategy: sources in,
tables out
    ...
    return signals, book, risk
```

```
# live
signals, book, risk = build_graph(live_trades, live_quotes)

# replay at real speed
rp = TableReplayer(start, end)
signals, book, risk = build_graph(rp.add_table(hist_trades, "Timestamp"),
                                  rp.add_table(hist_quotes, "Timestamp"))
rp.start()

# fast backtest on our own clock
with ctx_bound_to_event_driven_graph:
    trades, pub_t = table_publisher("bt_trades", schema,
update_graph=UpdateGraph(j_update_graph=ug))
    signals, book, risk = build_graph(blink_to_append_only(trades), ...)
for slice_ in history_slices:                    # e.g. one second of history
at a time
    pub_t.add(slice_)
    ug.requestRefresh()                           # one checkpoint; returns
when the step is done
```

What Deephaven does not give you, compared with a backtesting platform:

- **No fill model, brokerage model, fees or slippage.** Orders are rows; simulating fills is your table logic.
- **No portfolio or securities objects.** Positions and P&L are tables you build.
- **Time functions do not follow your clock** in Community replay or event-driven backtests. Use the data's own timestamps in formulas, not `now()`, and drive scheduled logic from data, not `time_table`.
- **Look-ahead is still possible.** Streaming replay is safe by construction, but a join against a full historical table (reference data, end-of-day marks) can see the future. Point-in-time joins (`aj`) avoid it.
- **External side effects** in listeners (sending orders, HTTP calls) run during a backtest too. Route them through one function you swap out.

A worked trading and risk engine

One function holds the whole strategy and risk engine as tables. We ran it live and as a backtest on the same 900 quotes. Both ended with an identical book and desk risk, and the backtest was 12 times faster. The full program is `examples/06_trading_engine.py`.

This is its engine:

```
def build_engine(quotes, limits, desks):
    q   = quotes.update("Mid = (Bid + Ask) / 2")
    ind = q.update_by([ema_tick(5, "Fast = Mid"), ema_tick(20, "Slow =
Mid")], by="Sym")
    sig = ind.update("Target = Fast > Slow ? 100L : -100L")                #
desired position
    orders = sig.update_by(delta("OrderQty = Target"), by="Sym") \
                .update("OrderQty = isNull(OrderQty) ? Target : OrderQty") \
                .where("OrderQty != 0") \
                .update(["FillPx = OrderQty > 0 ? Ask : Bid", "Notional =
OrderQty * FillPx"])
    positions = orders.agg_by([agg.sum_(["Pos = OrderQty", "Cost =
Notional"]), agg.count_("Trades")], by="Sym")
    book = positions.natural_join(q.last_by("Sym"), on="Sym", joins="Mid") \
                    .update(["Mtm = Pos * Mid", "PnL = Mtm - Cost", "Gross =
abs(Mtm)"])
    risk = book.natural_join(desks, on="Sym", joins="Desk") \
               .agg_by([agg.sum_(["Gross", "PnL"]), agg.count_("Names")],
by="Desk") \
               .natural_join(limits, on="Desk", joins="Limit") \
               .update("Breach = Gross > Limit")
    return {"signals": sig, "orders": orders, "book": book, "risk": risk}
```

The two ways of running it differ only in where the quotes come from and which clock steps the graph:

```
# live: a feed thread pushes quotes; the default ~1 s clock steps the graph
quotes, pub = table_publisher("live_quotes", schema)
engine = build_engine(blink_to_append_only(quotes), limits_input_table,
desks)
listen(engine["orders"], route_orders)              # the only per-cycle
Python

# backtest: same function, our clock, one checkpoint per historical slice
with ctx_on_event_driven_graph:
    quotes, pub = table_publisher("bt_quotes", schema,
update_graph=UpdateGraph(j_update_graph=ug))
    engine = build_engine(blink_to_append_only(quotes), limits, desks)
for rows in history:
    pub.add(slice_table(rows)); ug.requestRefresh()
```

Run	Wall time	Router calls	Orders	Final book and risk
Live (default clock)	5.76 s	4	33	identical
Backtest (our clock)	0.46 s	28	33	identical

The router-call counts show the clocks at work. Live, one cycle collected many slices, so 33 orders arrived in 4 batches. In the backtest every slice was its own checkpoint, so each slice that traded produced its own call. At the end the Tech desk was over its 15,000 limit (gross 20,833) and Semis was not (10,475).

What a production engine adds on top of this skeleton:

- real feeds behind `table_publisher` or Kafka
- `aj` joins for point-in-time reference data
- a real fill and fee model
- order routing in the listener, kept short and wrapped against exceptions
- `limits` as an input table the risk desk edits from a deephaven.ui dashboard
- the engine run as a PQ, its `book` and `risk` shared to other PQs by `pq://` URI

Deephaven vs. QuantConnect LEAN

LEAN is a trading framework: it owns time, securities, orders and fills, and you fill in callbacks. Deephaven is a data engine: it owns tables, recomputation and distribution, and you supply the trading model. That is why the LEAN docs read as a recipe and these read as a toolkit. Every LEAN concept has a Deephaven way to build it, but about half have no built-in equivalent.

LEAN concept	LEAN syntax	Deephaven equivalent	Built in?
Algorithm class	`class MyAlgo(QCAlgorithm)`	a script, or an `app.d/*.app` application, that runs once	yes
Setup	`initialize()`	the script body: declare sources and tables	yes
Per-slice callback	`on_data(self, slice)`	usually none: the graph updates itself; when needed, `merged_listen([...], fn)` once per cycle	yes

LEAN concept	LEAN syntax	Deephaven equivalent	Built in?
Time frontier, slice	the engine's time slice	an update-graph step (cycle); `requestRefresh()` on your own clock	yes
Subscriptions	`add_equity("SPY", Resolution.MINUTE)`	a source table: Kafka, `table_publisher`, `db.live_table`	yes, but you wire the feed
Consolidators	`consolidate("SPY", timedelta(minutes=5), fn)`	`update("Bin = lowerBin(Timestamp, 5 * MINUTE)").agg_by(..., by= ["Sym", "Bin"])`	yes
Indicators	`self.ema("SPY", 20)`	`update_by(ema_tick(20, ...), by="Sym")`, `rolling_*`	yes
Universe selection	`add_universe(fn)`	`where(...)`, `partition_by("Sym")` over every symbol at once	yes
Scheduled events	`schedule.on(date_rules..., time_rules..., fn)`	`time_table` + a listener, or your own loop	partly
Warm-up	`set_warm_up(20)`	load a history table and `merge` it ahead of the live feed	you build it
Portfolio, securities, cash	`self.portfolio["SPY"].quantity`	tables you build (positions, P&L)	**no**
Orders	`market_order("SPY", 100)`	rows in an orders table; a listener routes them	**no**
Brokerage, fill, fee, slippage models	`set_brokerage_model(...)`	table logic you write	**no**
Algorithm framework (Alpha, Portfolio Construction, Risk, Execution)	`set_alpha(...)` etc.	your own functions of tables	**no**
History requests	`self.history("SPY", 100)`	`db.historical_table(...)`, parquet, any table	yes
Backtest and live with the same code	built in	swap sources (replay or your clock) or an Enterprise replay query	yes, with the rule in Backtesting

LEAN concept	LEAN syntax	Deephaven equivalent	Built in?
Charts and UI	`plot(...)`	plotly-express, deephaven.ui dashboards, any client	yes, much richer
Sharing results with other apps	none	every named table is live to every client and other workers	yes, a core strength

Where Deephaven is stronger:

- **Cross-sectional work** over thousands of symbols at once, such as joins against positions, reference data and limits.
- **Live risk roll-ups** that stay current as any input changes.
- **Sharing:** many teams and UIs see the same live tables.
- **Ad-hoc inspection** of any table in a running app.

Where it is weaker for an algorithm framework:

- **No trading domain model:** no orders, fills or portfolio, and no brokerage integration.
- **Python per-row and per-cycle callbacks are the costliest parts** of the graph (about 195 ms per interval in our runs, against 3 to 36 ms for Java operations).
- **About 1 s default latency** for externally pushed rows, unless you shorten the cycle or step your own graph.
- **Time helpers follow the wall clock** in Community replay.
- **Testing and lifecycle hygiene are yours:** reloads, stuck listeners, shutdown.

The practical split for our firm: use Deephaven for the data and risk half (signals as tables, positions, P&L, limits, dashboards, sharing). Keep the order, fill and broker half in our algorithm framework, feeding Deephaven through `table_publisher` and reading its tables back. Or build that half once as a shared table library.

Source for LEAN: Algorithm engine.

From toolbox to recipes: the dhapp application layer

`dhapp` turns the toolbox into five steps: declare sources, write stages from recipes, add a few actions, surface tables, then start in a mode. It runs on `dhfw`, so the same app works on Community and Enterprise. Tested: the worked engine rebuilt on `dhapp` passed 8 of 8 checks (`examples/07_app_framework.py`). The live and backtest runs gave an identical book and risk, matching the hand-written rung-6 numbers exactly.

The full reference (quick start, API, environment variables, deployment and a cookbook) is in `dhapp and dhfw: Framework Reference`. This section is the summary.

```
app = App("mm", catalog)
app.source("quotes", schemas.QUOTES, history=load_quotes)   # pushed live;
replayed in backtest
app.source("limits", catalog="desk_limits")                 # wherever the
catalog says it lives

@app.stage                                                     # parameter
names ARE table names
def orders(quotes):
    return
recipes.fill_at_touch(recipes.target_to_orders(recipes.ema_cross(quotes)))

@app.stage
def risk(orders, quotes, desks, limits):
    return
recipes.risk_rollup(recipes.mark_to_market(recipes.positions(orders),
quotes), desks, limits)

@app.action(on="orders", services=["broker"])               # the only per-
cycle Python, supervised
def route(rows, broker): broker.send(rows)

app.service("broker", live=OrderGateway(), sim=RecordingBroker())
app.surface("risk", "orders", "limits")
app.start("live")            # or "backtest" (then app.run_history()) or
"test" (app.feed + app.step)
mm_dashboard = dashboard(app, kpis={"risk": ["Gross", "PnL"]})
```

How it answers each worry:

Worry	What dhapp does	Tested result
Too many options	Four roles (source, stage, action, surface) and recipes in `dhapp.recipes`: `mid`, `bars`, `ema_cross`, `target_to_orders`, `fill_at_touch`, `positions`, `mark_to_market`, `risk_rollup`. A stage is a plain function, so the full table API is always available	The rung-6 engine became 3 stages and 1 action
Building a dashboard	`dashboard(app)` makes one panel per surfaced table, optional KPI cards, and a status panel. `kpi_row`, `table_panel` and `status_panel` are the building blocks for your own	It rendered live in the browser. Editing a limit in the grid cleared a breach at the next cycle
Community vs. Enterprise	Everything that differs goes through `dhfw`; engine and UI code is not wrapped	The app ran on Community; Enterprise forwarding is untested here
Remote tables that become local on deployment	`Catalog` maps a logical name to a binding from `DHFW_PLANT` and `DHFW_PQ`: a local global when deployed inside the owning PQ, `remote_table` on the same plant, `for_remote_cluster` on another plant, `dhfw`'s mapping on a laptop	One `desk_limits` entry resolved three ways as the location changed
Persistence and your own tables	`dhapp.store`: `snapshot` (user table), `daily_partition` (history), `intraday_log` (Intraday), `state_table` (editable, keyed)	Snapshot and daily partitions round-tripped; the state table drove the dashboard

Worry	What dhapp does	Tested result
Trusting what runs	Actions are wrapped: an exception is recorded in `app.health` and the action keeps running. Surfaces are published as `<app>_<name>` fields	A failing action logged 1 error in 1 call while routing carried on
Testing streaming logic	`test` mode: `app.feed(...)`, then `app.step()`, then assert on `app.table(...)`, with no sleeps	A repeated target produced no second order

Rules and limits:

- **Frozen in backtest and test.** Catalog and live-factory sources tick on the default clock, so they are snapshotted in backtest and test modes. One graph cannot mix two clocks.
- **Backtest speed.** `run_history` replays history sources in time buckets, one checkpoint per bucket: 900 quotes in 150 buckets took 1.3 s.
- **Laptop as a client of a real plant** is not covered yet. On a laptop, remote names map to local Community servers. Reaching a real plant from a laptop needs the Core+ client and plant credentials.
- **Plant behaviour is unconfirmed.** On Enterprise, `state_table` and `intraday_log` persist; on Community they live only in memory. Both paths go through `dhfw`, and the Enterprise side is untested.

Testing on three tiers, and one API for both runtimes

Write against the Enterprise API. A thin wrapper, `dhfw`, forwards to Enterprise on the plant and emulates the same calls with Community constructs on a laptop, so application code never checks which runtime it is in. Tested: all 12 emulation checks pass on Community 42.6 (`tests/test_dhfw_community.py`). The Enterprise forwarding is untested here because we have no plant access; it calls the documented Core+ functions directly.

Tier	Runs on	Tests	How
Unit	embedded Community server in a test process	query logic on fixture tables	pytest, or a probe script like ours; `dhfw` with a temp data root
Local integration	Community server (pip or Docker)	ticking behaviour, listeners, UI, clients, threads, replay	our `run_server.py` with `dhfw` emulating `db` and `remote_table`
Plant	Enterprise Core+ worker	real data, PQ lifecycle, ACLs, cross-PQ sharing	`run_local_script` or a dev PQ; `dhfw` forwards to the real APIs

The Enterprise local-development guide covers only the unit tier, for Java and Groovy (Gradle, Core+ jars, a Mockito-mocked `db`). Our setup covers the other two, and `dhfw` makes the same code run in all three.

What the wrapper intercepts. Detection is automatic: Enterprise when `deephaven_enterprise` imports, overridable with `DHFW_RUNTIME`.

Enterprise API	Community emulation in `dhfw`	Limits on Community
`db.historical_table(ns, t)`	parquet under `DHFW_DATA_ROOT/<ns>/<t>/` with `Date=...` partitions	no schema service, merges or ACLs
`db.live_table(ns, t)`	an empty live table with the historical schema, which ticks as rows are appended; or a registered local feed (`db.register_live`)	in-process only
Replay query (`ReplayScript`)	`DHFW_REPLAY_DATE=YYYY-MM-DD`: `live_table` replays that partition through `TableReplayer`; `dhfw.today()` returns the replay date	1x speed only; `now()` in formulas stays wall time
`db.add_unpartitioned_table`, `add_table_partition`, `delete_*`	parquet writes and deletes	none significant

Enterprise API	Community emulation in `dhfw`	Limits on Community
`db.append_live_table`, `system_table_logger.log_table(_incremental)`	appends to the in-process live table (a listener for the incremental form)	not persisted; other processes cannot see it
`db` input tables	in-memory keyed input tables	lost on restart
`remote_table.in_local_cluster(q, t).subscribe(...)`, `pq://q/scope/t`	`dh+plain` Barrage subscription to a server mapped by `DHFW_PQ_MAP` (or a global in this worker)	filters and columns apply after transfer; no reconnect policy
`controller_import.meta_import`, `exec_script`	imports from a local checkout of the controller repository (`DHFW_CONTROLLER_ROOT`)	not the controller's Git revision
`edge_acl`, `acl_generator`	no-op with a warning	no permissions enforced
`auth_context`	`DHFW_USER` or the OS user	no Enterprise identity
Anything else on `db`	raises `UnsupportedOnCommunity` with a pointer to the limits	explicit, never silent

What is not wrapped, because it is the same engine on both: table operations, listeners, update graphs and execution contexts, `table_publisher`, deephaven.ui and plotly-express. The one exception is the version. The plant's Core+ worker may run a different Core version, and `dhfw.runtime()` warns when `DHFW_PLANT_CORE_VERSION` differs from the local one. Some things cannot be emulated at all: PQ scheduling, restarts and failover, query servers, merge and validation jobs, and Enterprise authentication.

Application code looks identical on both runtimes:

```
from dhfw import db, remote_table, today

quotes = db.live_table("MarketUs", "QuoteStock").where(f"Date ==
`{today()}`")
limits = remote_table("RiskLimits", "desk_limits").subscribe()
engine = build_engine(quotes, limits, desks)          # the graph: no runtime
checks anywhere
```

Native-Windows gotchas the wrapper and its users must handle (the first found while building it):

- Deephaven's parquet read and write parse paths as URIs, so `C:\...` fails with a URISyntaxException. `dhfw` passes `file:///` URIs.
- Integer type hints on Python functions are rejected (see Extending).
- `query_library.import_static` cannot see jars added with `extra_classpath` (see Extending).

Evidence and sources

Everything marked Tested ran on Deephaven Community 42.6 (pip server, native Windows, Java 17) in the `deephaven-compare` repo. Earlier facts are in `VERIFIED_FACTS.md`.

Claim	Test or source	Result
Java function in a formula	`tests/probe_framework_constructs.py` with `tests/java/com/acme/Risk.java`	`com.acme.Risk.dv01(...)` returned 7.4625 and -3.73125
`import_static("name")` fails for an added jar	same	ClassNotFound, while `jpy.get_type` loaded it
Python function in a formula	same	float hints gave a `double` column; no hints gave `PyObject`; `int` and `np.int64` hints were rejected
`function_generated_table`	same	ran 3 times in 1.5 s at a 200 ms setting (paced by the 1 s cycle)
`ring_table`	same	held exactly 5 rows of a 10-per-second feed
Keyed input table	same	upsert by key replaced Rates 1e6 with 2e6; delete by key left 1 row
`TableReplayer`	same	released 20 rows over about 4 s; unchanged graph code; `now()` stayed wall time
Event-driven backtest	same	200 slices in 0.137 s; one step and one callback each
Same engine live and as a backtest	`examples/06_trading_engine.py`	identical final book and risk; 5.76 s vs 0.46 s

Claim	Test or source	Result
Enterprise replay query	Create a replay query	Docs
Java stream sources	StreamToBlinkTableAdapter Javadoc	Docs
Java classes on the classpath	Install and use Java packages	Docs
LEAN engine model	QuantConnect: Algorithm engine	Docs